Alternative Polyadenylation in scRNA-Seq

Alternative polyadenylation (APA) is a biological process that occurs during the maturation of messenger RNA (mRNA), where different polyadenylation signals are used within the same gene. This means a single DNA sequence can generate multiple versions of mRNA with different lengths at the 3′ end. These variations can affect RNA stability, cellular localization, and how the RNA is translated into protein. As a result, APA plays an important role in regulating gene expression and may be associated with processes such as cell differentiation and diseases like cancer.

The SCAPE-APA (Single Cell Analysis of Polyadenylation Events) is a computational package developed to identify and quantify alternative polyadenylation events in scRNA-seq data. It allows researchers to study how different cells use distinct polyadenylation sites in their genes, revealing additional layers of regulation that are not captured by classical gene expression analysis.

In this notebook, we will use SCAPE-APA within the Google Colab environment to estimate alternative polyadenylation events from scRNA-seq data. The goal is to offer a practical and intuitive introduction to this type of analysis, demonstrating how APA contributes to regulatory diversity across individual cells.

Conda

Conda is a package for environment manager used primarily in data science and bioinformatics. It makes it easy to install software and libraries, even those with complex dependencies.

A Conda virtual environment is an isolated space where you can install specific versions of packages without interfering with other installations on your system. This avoids conflicts and ensures reproducibility of analyses.

In [ ]:
```
# Installing the condalab package, allowing you to use Conda in Google Colab, along with its p
#Let this finish before moving on
!pip install -q condacolab
import condacolab
condacolab.install()
```

In [ ]:
```
%%capture
# Installation of numerical packages and tools for data visualization.
!conda install -y -c conda-forge -c bioconda -c defaults \
    numpy scipy pandas matplotlib \
    click tomli-w requests psutil \
    bedtools pybedtools pysam gffutils

#Install scape
!pip install taichi scape-apa
!apt-get install -y samtools
%env MPLBACKEND=Agg
```

In [ ]:
```
# Verifying SCAPE-APA installation.
!scape --help
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.11
[Taichi] Starting on arch=cuda
GPU is available

This software is affiliated with the following paper:
SCAPE-APA: a package for estimating alternative polyadenylation events from scRNA-seq data
Guangzhao Cheng[1], Tien Le[1], Ran Zhou, Lu Cheng[+]
BioRxiv
2024
https://doi.org/10.1101/2024.03.12.584547

Usage: scape [OPTIONS] COMMAND [ARGS]...

  An analysis framework for estimating alternative polyadenylation events from
  single cell RNA-seq data. Current version: 1.0.0

Options:
  --help  Show this message and exit.

Commands:
  cal_exp_pa_len      INPUT: - output_dir: path to output_dir folder (to...
  ex_pa_cnt_mat       INPUT: - output_dir: path to output_dir folder
  gen_utr_annotation
  infer_pa            INPUT: - pkl_input_file: file path (pickle)...
  merge_pa
  prepare_input
```

GitHub

GitHub is an online platform where developers can store, share, and collaborate on code projects using the Git version control system.

The git clone command is used to copy an entire repository from GitHub (or another Git server) to your local repository. It downloads all of the files, version history, and project structure, allowing you to work locally. Let's do this like SCAPE-APA

In [1]:
```
# Getting the files from the GitHub repository.
!git clone https://github.com/chengl7-lab/scape.git
```

```
Cloning into 'scape'...
remote: Enumerating objects: 275, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 275 (delta 2), reused 3 (delta 0), pack-reused 262 (from 1)
Receiving objects: 100% (275/275), 9.01 MiB | 5.82 MiB/s, done.
Resolving deltas: 100% (128/128), done.
```

In [2]:
```
#List of files inside the "toy example" folder.

!ls -lh scape/examples/toy-example/
```

```
total 12M
-rw-r--r-- 1 jovyan users 894K Feb  7 00:34 barcode_index.csv
-rw-r--r-- 1 jovyan users 135K Feb  7 00:34 barcodes.tsv.gz
-rw-r--r-- 1 jovyan users 410K Feb  7 00:34 cluster_wrt_CB.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.gene.pa.len.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.utr.pa.len.csv
-rw-r--r-- 1 jovyan users 3.9M Feb  7 00:34 example.bam
-rw-r--r-- 1 jovyan users  50K Feb  7 00:34 example.bam.bai
-rw-r--r-- 1 jovyan users 4.8M Feb  7 00:34 GRCh38_98.csv
drwxr-sr-x 3 jovyan users 4.0K Feb  7 00:34 pkl_input
drwxr-sr-x 2 jovyan users 4.0K Feb  7 00:34 pkl_output
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.gene.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 00:34 res.gene.pkl
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.utr.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 00:34 res.utr.pkl
```

In [3]:
```
# Download annotation file "Mus_musculus.GRCm39.113.chr.gff3"
# wget is for downloading
# -O to define where the file goes and what name
!wget -O scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3.gz \
"https://ftp.ensembl.org/pub/release-113/gff3/mus_musculus/Mus_musculus.GRCm39.113.chr.gff3.gz
```

```
--2026-02-07 00:34:36--  https://ftp.ensembl.org/pub/release-113/gff3/mus_musculus/Mus_musculu
s.GRCm39.113.chr.gff3.gz
Resolving ftp.ensembl.org (ftp.ensembl.org)... 193.62.193.169, 193.62.193.169, 193.62.193.169,
...
Connecting to ftp.ensembl.org (ftp.ensembl.org)|193.62.193.169|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 37011687 (35M) [application/x-gzip]
Saving to: ‘scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3.gz’

scape/examples/toy- 100%[===================>]  35.30M  5.62MB/s    in 7.3s

2026-02-07 00:34:44 (4.86 MB/s) - ‘scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3.
gz’ saved [37011687/37011687]
```

In [4]:
```
# Uncompress file
!gunzip  scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3.gz
```

In [5]:
```
# Checking internal content.
!head -n 20 scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3
```

```
##gff-version 3
##sequence-region   1 1 195154279
##sequence-region   10 1 130530862
##sequence-region   11 1 121973369
##sequence-region   12 1 120092757
##sequence-region   13 1 120883175
##sequence-region   14 1 125139656
##sequence-region   15 1 104073951
##sequence-region   16 1 98008968
##sequence-region   17 1 95294699
##sequence-region   18 1 90720763
##sequence-region   19 1 61420004
##sequence-region   2 1 181755017
##sequence-region   3 1 159745316
##sequence-region   4 1 156860686
##sequence-region   5 1 151758149
##sequence-region   6 1 149588044
##sequence-region   7 1 144995196
##sequence-region   8 1 130127694
##sequence-region   9 1 124359700
```

In [6]:
```
# check file lines
!wc -l scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3
```

```
2513492 scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3
```

UTR Region Annotations

UTRs are untranslated regions of messenger RNA, located at the 5′ and 3′ ends of the gene. APA usually occurs at the 3′ end, within the 3′UTR. This means that correctly defining where these regions are located is essential to accurately detect the different APA events that can occur in a gene.

The `gen_utr_annotation` command is part of the SCAPE-APA analysis flow, and its main objective is to generate annotations of UTR (Untranslated Regions) regions from a file in .gff3 format, which contains the genomic annotations of a species.

- What does the command do in practice?

1. It reads the .gff3 file, which contains information about the structure of the genes.
2. It identifies and extracts the 3′UTR regions of each transcript based on these annotations.
3. Generate an output file with these regions, which will be used by SCAPE-APA to detect the locations where different polyadenylation signals appear.

NOTE: This takes about 1 hour on GPU or 2 hours on CPU. Skip this step if you are using the example files.

In [ ]:
```
# scape gen_utr_annotation = Call SCAPE with the "gen_utr_annotation" command.
# --gff_file = ".gff3" file.
# --output_dir = Output path of the generated file.
```

```
# --res_file_name = Name of the generated file, resulting in ".csv" format.

!scape gen_utr_annotation \
--gff_file scape/examples/toy-example/Mus_musculus.GRCm39.113.chr.gff3 \
--output_dir scape/examples/toy-example/ \
--res_file_name example_annotation
```

Run this to get the ready-to-use output file

In [77]:
```
!wget -O scape/examples/toy-example/example_annotation.csv \
"https://raw.githubusercontent.com/integrativebioinformatics/scNotebooks/refs/heads/main/scNot
```

```
--2026-02-07 02:06:59--  https://raw.githubusercontent.com/integrativebioinformatics/scNotebook
s/refs/heads/main/scNotebooks-Resources/example_annotation.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.10
8.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... con
nected.
HTTP request sent, awaiting response... 200 OK
Length: 5653844 (5.4M) [text/plain]
Saving to: ‘scape/examples/toy-example/example_annotation.csv’

scape/examples/toy- 100%[===================>]   5.39M  18.4MB/s    in 0.3s

2026-02-07 02:07:00 (18.4 MB/s) - ‘scape/examples/toy-example/example_annotation.csv’ saved [56
53844/5653844]
```

Prepare scRNA-seq data

The `prepare_input` command is an essential step in the SCAPE-APA pipeline and aims to prepare scRNA-seq data for the detection of alternative polyadenylation (APA) events.

By focusing only on reads located in UTR regions, this step reduces noise and focuses the analysis on the areas where APA events occur. This improves the accuracy and efficiency of detection, allowing a more reliable estimation of the different polyadenylation sites in individual cells.

- What it does:

1. Processes the .bam file, which contains the alignments of single-cell RNA-seq reads.
2. Filters and selects only those reads that align to UTR regions (mainly 3’UTRs) previously annotated with the gen_utr_annotation command.
3. Consolidates the relevant data into a format suitable for subsequent SCAPE-APA analyses.

NOTE: This takes about 7 minutes to finish on GPU. Skip this step if you are using the example files.

In [75]:
```
# scape prepare_input = Command to prepare data from a ".bam" file.
# --utr_file = Path to the UTR annotation file generated above.
# --cb_file = Barcode file to identify cells in scRNA-seq data.
# --bam_file = ".bam" file of aligned reads.

!scape prepare_input --utr_file scape/examples/toy-example/example_annotation.csv \
--cb_file scape/examples/toy-example/barcodes.tsv.gz \
--bam_file scape/examples/toy-example/example.bam \
--output_dir scape/examples/toy-example/
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.6
[Taichi] Starting on arch=cuda
GPU is available

This software is affiliated with the following paper:
SCAPE-APA: a package for estimating alternative polyadenylation events from scRNA-seq data
Guangzhao Cheng[1], Tien Le[1], Ran Zhou, Lu Cheng[+]
BioRxiv
2024
https://doi.org/10.1101/2024.03.12.584547

Processing  scape/examples/toy-example/example.bam
Read existing index file for barcode scape/examples/toy-example/barcode_index.csv
Done 0 files in 6.555305869733335 minutes.
```

In [9]:
```
# check directory
!ls -lh scape/examples/toy-example/
```

```
total 433M
-rw-r--r-- 1 jovyan users 894K Feb  7 00:34 barcode_index.csv
-rw-r--r-- 1 jovyan users 135K Feb  7 00:34 barcodes.tsv.gz
-rw-r--r-- 1 jovyan users 410K Feb  7 00:34 cluster_wrt_CB.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.gene.pa.len.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.utr.pa.len.csv
-rw-r--r-- 1 jovyan users 5.4M Feb  7 00:35 example_annotation.csv
-rw-r--r-- 1 jovyan users 3.9M Feb  7 00:34 example.bam
-rw-r--r-- 1 jovyan users  50K Feb  7 00:34 example.bam.bai
-rw-r--r-- 1 jovyan users 4.8M Feb  7 00:34 GRCh38_98.csv
-rw-r--r-- 1 jovyan users 416M Aug 18  2024 Mus_musculus.GRCm39.113.chr.gff3
drwxr-sr-x 3 jovyan users 4.0K Feb  7 00:34 pkl_input
drwxr-sr-x 2 jovyan users 4.0K Feb  7 00:34 pkl_output
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.gene.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 00:34 res.gene.pkl
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.utr.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 00:34 res.utr.pkl
```

Samtools

Samtools is a collection of command-line tools used to manipulate sequence alignment files, especially in SAM (Sequence Alignment/Map) and BAM (compressed version of SAM) formats.

Let's install it below:

In [ ]:
```
# Viewing ".bam" file with samtools.
```

```
!samtools view scape/examples/toy-example/example.bam | head -n 10
```

Samtools allows you to create an index of the bam file, which allows bioinformatics tools to quickly access specific regions of the genome within that file. This is essential, for example, for viewing alignments in genome browsers or for analyses that involve extracting specific regions.

In [76]:
```
# Make index
!samtools index scape/examples/toy-example/example.bam
```

In [12]:
```
#Lists all files in the specified directory
# The pipe (|) is an operator used to chain commands,
# allowing the output of one command to be used directly as input for the next
# | grep ".pkl" = filters the output of the previous command and shows only the lines that con

!ls -lh scape/examples/toy-example/ | grep ".pkl"
```

```
drwxr-sr-x 3 jovyan users 4.0K Feb  7 00:34 pkl_input
drwxr-sr-x 2 jovyan users 4.0K Feb  7 00:34 pkl_output
-rw-r--r-- 1 jovyan users 622K Feb  7 00:34 res.gene.pkl
-rw-r--r-- 1 jovyan users 622K Feb  7 00:34 res.utr.pkl
```

Inferring APA events

With SCAPE-APA it is possible to infer how the cell uses different RNA termination signals, a fundamental process for understanding gene regulation at the single-cell level. Detecting these events allows us to investigate more refined expression patterns and their possible implications for biological processes or diseases.

For this, `infer_pa` is used in the previously annotated 3'UTR regions.

What it does:

1. Analyzes the input data (filtered in the prepare_input step) to detect alternative polyadenylation sites (APA sites) within the UTR regions.
2. Identifies at which positions different transcripts of the same gene are being polyadenylated, allowing us to infer the diversity of post-transcriptionally regulated isoforms.

In [19]:
```
# Command to run the inference model and detect polyadenylation sites.
!scape infer_pa \
--pkl_input_file scape/examples/toy-example/pkl_input/example.100.1.1.input.pkl \
--output_dir scape/examples/toy-example/ \
--toml_para_file scape/tutorial/default_config.toml
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.6
[Taichi] Starting on arch=cuda
GPU is available

This software is affiliated with the following paper:
SCAPE-APA: a package for estimating alternative polyadenylation events from scRNA-seq data
Guangzhao Cheng[1], Tien Le[1], Ran Zhou, Lu Cheng[+]
BioRxiv
2024
https://doi.org/10.1101/2024.03.12.584547

Parameter file scape/tutorial/default_config.toml loaded.
watch_dog_flag = False
re_run_mode = True
debug = False
n_max_apa = 5
n_min_apa = 1
utr_length = 2000
min_LA = 20
max_LA = 150
mu_f = 300
sigma_f = 50
min_pa_gap = 100
max_beta = 70
theta_step = 9
beta_step = 5
min_ws = 0.05
max_unif_ws = 0.15

start inferring APA events from input pickle file = scape/examples/toy-example/pkl_input/exampl
e.100.1.1.input.pkl. Output file = scape/examples/toy-example/pkl_input/example.100.1.1.input.p
kl
open file as file handler
start each UTR region
/opt/conda/lib/python3.11/site-packages/scape/apa_core.py:305: RuntimeWarning: All-NaN slice en
countered
  if np.isnan(np.nanmax(r_arr)):
----------Estimated Parameters K=5----------
K=5 L=4370 Last component is uniform component.
alpha_arr=[ 445 1327 2560 2965 4171]
beta_arr=[20. 60. 50. 45. 50.]
ws=[0.03 0.   0.01 0.15 0.8  0.01]
bic=883336.87
------------------------------

----------Estimated Parameters K=4----------
K=4 L=4370 Last component is uniform component.
alpha_arr=[ 445 2560 2965 4171]
beta_arr=[15. 50. 45. 50.]
ws=[0.03 0.01 0.15 0.8  0.01]
bic=883511.36
------------------------------

----------Estimated Parameters K=3----------
K=3 L=4370 Last component is uniform component.
alpha_arr=[ 445 2965 4171]
```

```
beta_arr=[15. 55. 50.]
ws=[0.03 0.15 0.8  0.02]
bic=885149.6
------------------------------

----------Estimated Parameters K=2----------
K=2 L=4370 Last component is uniform component.
alpha_arr=[2965 4171]
beta_arr=[45. 50.]
ws=[0.15 0.8  0.05]
bic=894972.0
------------------------------

----------Estimated Parameters K=1----------
K=1 L=4370 Last component is uniform component.
alpha_arr=[4171]
beta_arr=[50.]
ws=[0.85 0.15]
bic=948170.6
------------------------------

Remove components [0, 1, 2] with weight less than min_ws=0.05.
----------Final Result K=2----------
K=2 L=4370 Last component is uniform component.
alpha_arr=[2965 4171]
beta_arr=[45. 50.]
ws=[0.15 0.8  0.05]
bic=905323.5
------------------------------

Done 10:ENSG00000099194:1:100360634-100365126:+ in 1.2535322871000063 min.
start each UTR region
save result of 10:ENSG00000099194:1:100360634-100365126:+
```

Executing the `infer_pa` command This command is responsible for inferring alternative polyadenylation (APA) events from scRNA-seq alignments and previously annotated UTR regions.

Main command arguments:

- utr_file: .csv file with annotations of the UTR regions, generated in the gen_utr_annotation step.
- cb_file: File containing the cellular barcodes, which identify each cell individually.
- bam_file: .bam file with RNA reads aligned to the genome.
- output_dir: Directory where the output files will be saved.

Main inference parameters:

- chunksize: Number of UTRs analyzed at a time (useful for managing memory in large datasets).
- n_max_apa / n_min_apa: Maximum and minimum number of APA sites that the algorithm will detect per UTR.

- min_LA / max_LA: Minimum and maximum allowed size for alignment regions (adjusts noise and precision).
- mu_f / sigma_f: Mean and standard deviation of RNA fragment lengths (essential for modeling the data).
- min_pa_gap: Minimum distance between two distinct APA sites — avoids detecting false positives that are too close together.
- max_beta, beta_step: Controls the beta distribution, used in statistical modeling of the position of APA sites.
- theta_step: Range of theta values, another parameter of the mixture of distributions.
- min_ws / max_unif_ws: Controls the weights of the distributions in the modeling — essential for adjusting real events vs. noise.
- re_run_mode: If enabled (TRUE), allows repeating the analysis by overwriting previous results.
- fixed_run_mode: If disabled (FALSE), allows the algorithm to automatically adjust parameters for better inference.

Grouping events

The `merge_pa` command is used to group APA events detected in previous steps of SCAPE-APA, unifying results that belong to the same gene or the same UTR region.

What it does in practice:

1. Receives as input the inference results of APA events by cell or by UTR region (obtained via `infer_pa`).
2. Groups APA sites that occur in the same genomic region (for example, within the same 3'UTR or in a single gene).
3. Eliminates redundancies and generates a consolidated summary of the most representative cutoff points for each gene.

During inference, multiple APA sites can be detected in different cells or replicates. The `merge_pa` function allows unifying similar events, facilitating interpretation. Furthermore, it also reduces statistical noise and helps in the overall visualization of APA patterns per gene, which can be used in further analysis such as clustering, visualizations or association with cellular metadata.

In [55]:
```
# With "merge_pa", the polyadenylation sites inferred in the previous step are collected,
# #merging the sites within the same gene or UTR.

# Generates two "res.TYPE.pk1" files, where "TYPE" can be the gene sites
```

```
# or merged UTRs (gene - UTR).

!scape merge_pa --output_dir scape/examples/toy-example/
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.6
[Taichi] Starting on arch=cuda
GPU is available

This software is affiliated with the following paper:
SCAPE-APA: a package for estimating alternative polyadenylation events from scRNA-seq data
Guangzhao Cheng[1], Tien Le[1], Ran Zhou, Lu Cheng[+]
BioRxiv
2024
https://doi.org/10.1101/2024.03.12.584547

Done read model output_dir
Done read model input
[]
0.012003394000203116
```

In [41]:
```
# List files in toy-example
!ls -lh scape/examples/toy-example/
```

```
total 433M
-rw-r--r-- 1 jovyan users 894K Feb  7 00:34 barcode_index.csv
-rw-r--r-- 1 jovyan users 135K Feb  7 00:34 barcodes.tsv.gz
-rw-r--r-- 1 jovyan users 410K Feb  7 00:34 cluster_wrt_CB.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.gene.pa.len.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.utr.pa.len.csv
-rw-r--r-- 1 jovyan users 5.4M Feb  7 00:35 example_annotation.csv
-rw-r--r-- 1 jovyan users 3.9M Feb  7 00:34 example.bam
-rw-r--r-- 1 jovyan users  50K Feb  7 00:36 example.bam.bai
-rw-r--r-- 1 jovyan users 4.8M Feb  7 00:34 GRCh38_98.csv
-rw-r--r-- 1 jovyan users 416M Aug 18  2024 Mus_musculus.GRCm39.113.chr.gff3
drwxr-sr-x 3 jovyan users 4.0K Feb  7 00:34 pkl_input
drwxr-sr-x 2 jovyan users 4.0K Feb  7 00:46 pkl_output
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.gene.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 01:08 res.gene.pkl
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.utr.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 01:02 res.utr.pkl
```

In [56]:
```
# SCAPE-APA analysis to calculate the effective length of detected APA events

!scape cal_exp_pa_len \
--output_dir scape/examples/toy-example \
--res_pkl_file res.gene.pkl
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.6
[Taichi] Starting on arch=cuda
GPU is available

This software is affiliated with the following paper:
SCAPE-APA: a package for estimating alternative polyadenylation events from scRNA-seq data
Guangzhao Cheng[1], Tien Le[1], Ran Zhou, Lu Cheng[+]
BioRxiv
2024
https://doi.org/10.1101/2024.03.12.584547

Done calculating expected pa length each gene in 9.025366671266965e-06 min.
Done in 2.637624999503411e-05 min.
```

In [57]:
```
# List files in toy-example
!ls -lh scape/examples/toy-example/
```

```
total 433M
-rw-r--r-- 1 jovyan users   64 Feb  7 01:16 all_cell.gene.pa.len.csv
-rw-r--r-- 1 jovyan users 894K Feb  7 00:34 barcode_index.csv
-rw-r--r-- 1 jovyan users 135K Feb  7 00:34 barcodes.tsv.gz
-rw-r--r-- 1 jovyan users 410K Feb  7 00:34 cluster_wrt_CB.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 01:13 cluster_wrt_CB.gene.pa.len.csv
-rw-r--r-- 1 jovyan users   83 Feb  7 00:34 cluster_wrt_CB.utr.pa.len.csv
-rw-r--r-- 1 jovyan users 5.4M Feb  7 00:35 example_annotation.csv
-rw-r--r-- 1 jovyan users 3.9M Feb  7 00:34 example.bam
-rw-r--r-- 1 jovyan users  50K Feb  7 00:36 example.bam.bai
-rw-r--r-- 1 jovyan users 4.8M Feb  7 00:34 GRCh38_98.csv
-rw-r--r-- 1 jovyan users 416M Aug 18  2024 Mus_musculus.GRCm39.113.chr.gff3
drwxr-sr-x 3 jovyan users 4.0K Feb  7 00:34 pkl_input
drwxr-sr-x 2 jovyan users 4.0K Feb  7 00:46 pkl_output
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.gene.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 01:15 res.gene.pkl
-rw-r--r-- 1 jovyan users 147K Feb  7 00:34 res.utr.cnt.tsv.gz
-rw-r--r-- 1 jovyan users 622K Feb  7 01:02 res.utr.pkl
```

Viewing results

`gen_utr_annotation`

Pandas is a powerful library for data manipulation and analysis in Python. With it, you can work with tables (DataFrames) in a similar way to Excel spreadsheets or SQL tables. Let's use it to check the data

In [58]:
```
import pandas as pd

# Load the UTR annotation
df_utr = pd.read_csv("scape/examples/toy-example/example_annotation.csv")

# Display the first few rows of the table
df_utr.head()
```

Out[58]:

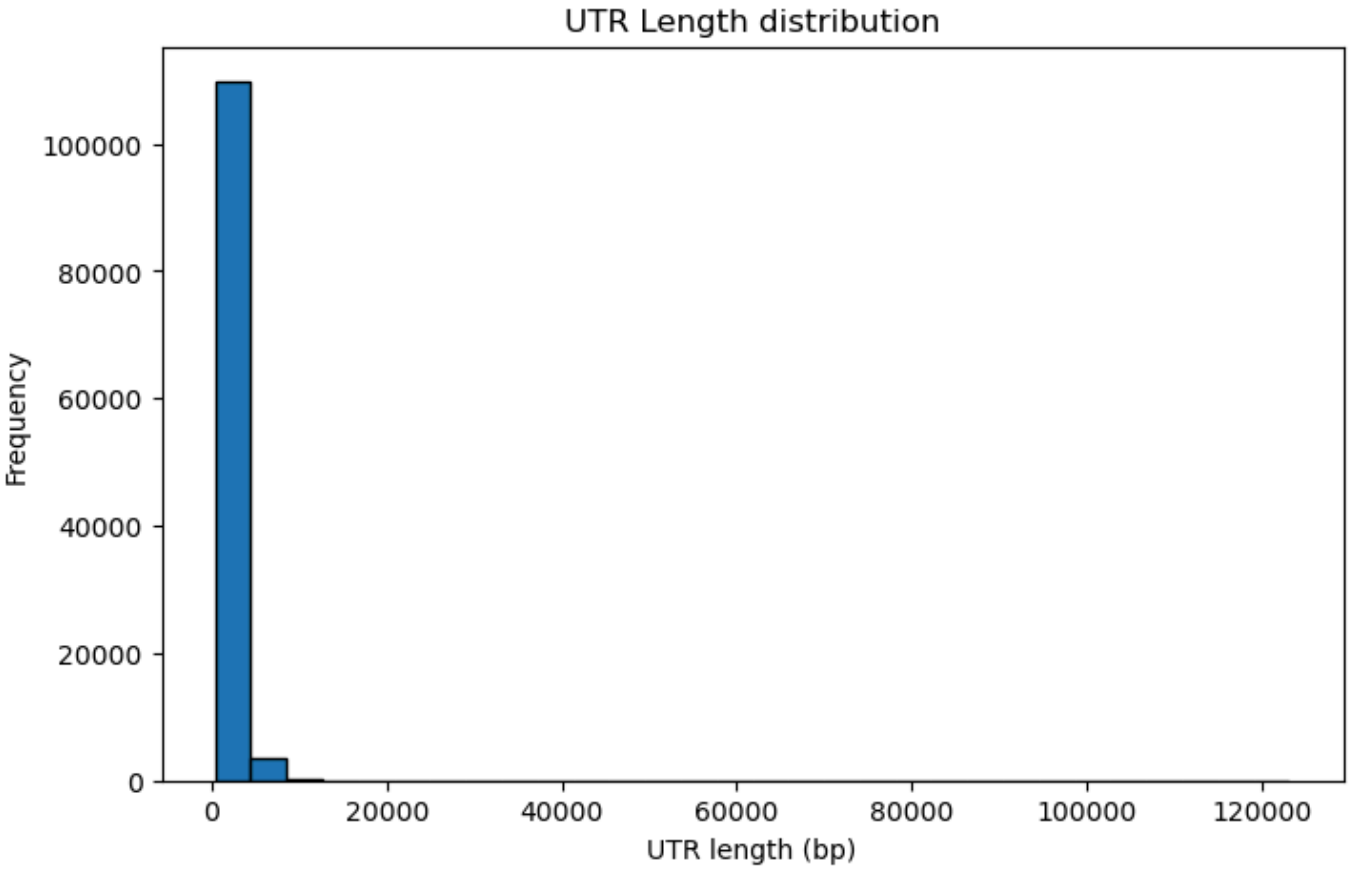
	chrom	start	end	strand	gene_id	gene_name	utr_id
0	1	3275824	3277840	-	ENSMUSG00000051951	Xkr4	1
1	1	3284405	3286544	-	ENSMUSG00000051951	Xkr4	2
2	1	4069480	4070140	-	ENSMUSG00000025900	Rp1	1
3	1	4360769	4363535	-	ENSMUSG00000025900	Rp1	2
4	1	4414069	4415122	-	ENSMUSG00000025900	Rp1	3

In [59]:

```
import matplotlib.pyplot as plt

# Calculate the length of each UTR
df_utr["length"] = df_utr["end"] - df_utr["start"]

# Plot a length histogram
plt.figure(figsize=(8, 5))
plt.hist(df_utr["length"], bins=30, edgecolor='black')
plt.xlabel("UTR length (bp)")
plt.ylabel("Frequency")
plt.title("UTR Length distribution")
plt.show()
```



In [60]:

```
# Run merge_pa
!scape merge_pa --output_dir scape/examples/toy-example \
--utr_merge True
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.6
[Taichi] Starting on arch=cuda
GPU is available

This software is affiliated with the following paper:
SCAPE-APA: a package for estimating alternative polyadenylation events from scRNA-seq data
Guangzhao Cheng[1], Tien Le[1], Ran Zhou, Lu Cheng[+]
BioRxiv
2024
https://doi.org/10.1101/2024.03.12.584547

Done read model output_dir
Done read model input
[]
0.011150401998747839
```

In [61]:
```python
# Check if "prepare_input" worked
# Expect ".pk1" files

import os

# List files
os.listdir("scape/examples/toy-example/pkl_input/")
```

Out[61]:
```
['example.100.1.1.input.pkl', 'example.log', '.ipynb_checkpoints']
```

In [62]:
```python
# Displaying parameters by "infer_pa" (res.utr.pkl)
import pickle
f=open("scape/examples/toy-example/res.utr.pkl", "rb")
print(pickle.load(f))
```

```
[Taichi] version 1.7.4, llvm 15.0.4, commit b4b956fd, linux, python 3.11.6
[I 02/07/26 01:16:34.503 99] [shell.py:_shell_pop_print@23] Graphical python shell detected, us
ing wrapped sys.stdout
[Taichi] Starting on arch=cuda
GPU is available
----------Final Result K=2----------
gene info: 10:ENSG00000099194:1:100360634-100365126:+
K=2 L=0 Last component is uniform component.
alpha_arr=[2965 4171]
beta_arr=[45. 50.]
ws=[0.16 0.84]
------------------------------
```

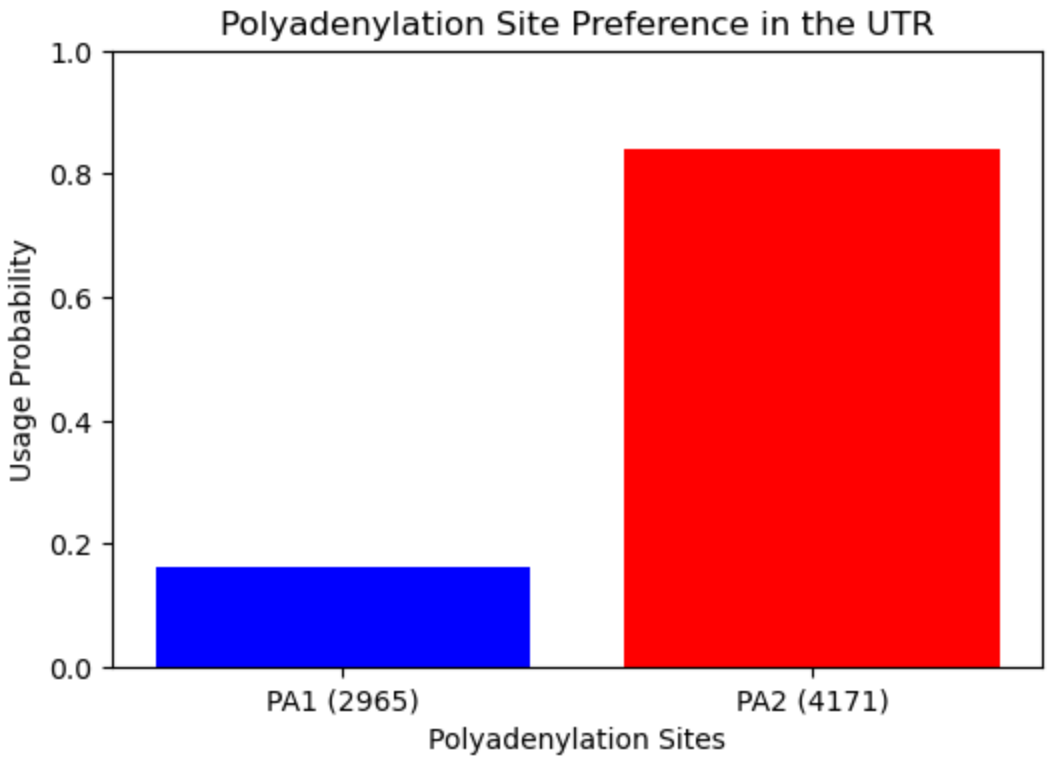
In [63]:
```python
import matplotlib.pyplot as plt

# PA Site Data
pa_sites = ["PA1 (2965)", "PA2 (4171)"]
usage_probs = [0.16, 0.84]

# Create Bar Chart
plt.figure(figsize=(6, 4))
plt.bar(pa_sites, usage_probs, color=['blue', 'red'])
plt.xlabel("Polyadenylation Sites")
plt.ylabel("Usage Probability")
```

```
plt.title("Polyadenylation Site Preference in the UTR")
plt.ylim(0, 1)

# Show plot
plt.show()
```



In [64]:
```
# merge_pa
# review data
with open("scape/examples/toy-example/res.gene.pkl", "rb") as f:
    try:
        merged_pa = pickle.load(f)
        print(merged_pa)
    except EOFError:
        print("The file res.gene.pkl is empty.")
```

```
----------Final Result K=2----------
gene info: 10:ENSG00000099194:1:100360634-100365126:+
K=2 L=0 Last component is uniform component.
alpha_arr=[2965 4171]
beta_arr=[45. 50.]
ws=[0.16 0.84]
------------------------------
```

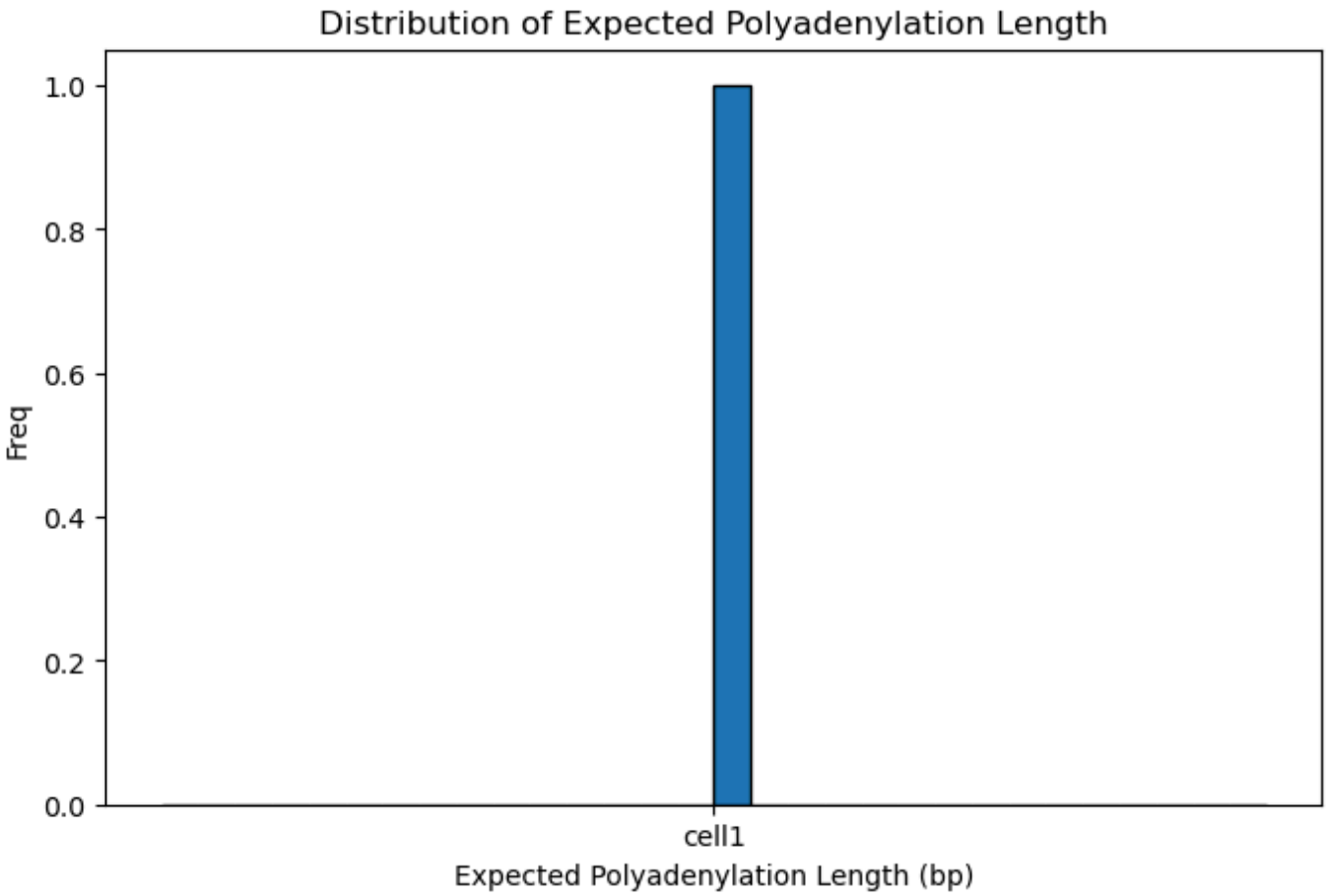
In [65]:
```
# cal_exp_pa_len
df_exp_len = pd.read_csv("scape/examples/toy-example/cluster_wrt_CB.gene.pa.len.csv")

# Show first lines
df_exp_len.head()
```

Out[65]:

	gene_id	cell_cluster	exp_length	num_pa
0	ENSG00000099194:1	cell1	8.555119	2

In [66]:
```
plt.figure(figsize=(8, 5))
plt.hist(df_exp_len.iloc[:, 1], bins=30, edgecolor='black')
plt.xlabel("Expected Polyadenylation Length (bp)")
plt.ylabel("Freq")
plt.title("Distribution of Expected Polyadenylation Length")
plt.show()
```



In [67]:
```
#ex_pa_cnt_mat

df_counts = pd.read_csv("scape/examples/toy-example/res.utr.cnt.tsv.gz", sep="\t")

# Show fisrt lines
df_counts.head()
```

Out[67]:

	pa_info	C1_AAACCTGAGACTCGGA-1	C1_AAACCTGAGAGCCTAG-1	C1_AA
0	10:100363599:45.0:+:1:ENSG00000099194:1	0.0	0.0	
1	10:100364805:50.0:+:2:ENSG00000099194:1	0.0	0.0	

2 rows × 33089 columns

In [68]:
```
plt.figure(figsize=(8, 5))
plt.hist(df_counts.iloc[:, 1], bins=30, edgecolor='black')
plt.xlabel("Number of Polyadenylation Events")
plt.ylabel("Frequency")
plt.title("Distribution of Polyadenylation Events")
plt.show()
```